

Back-end em python



Prof^a. Ma. Ana Clara Gomes

SOFTEX
PERNAMBUCO

 **Softex**

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO



Aula 30 — Introdução ao Padrão MVC + Análise de Sistema OO

Objetivos da Aula

- Compreender o que é MVC e por que ele é usado.
- Identificar responsabilidades (Model, View, Controller).
- Relacionar MVC com a estrutura de um projeto Python/Django





Aula 30 — Introdução ao Padrão MVC + Análise de Sistema OO



O que é MVC ?

MVC significa **Model–View–Controller**, um **padrão de arquitetura de software** usado para **organizar o código** de forma que **separe a lógica de negócios da interface com o usuário**. Isso facilita a **manutenção, testes e reutilização** do código.

Camada	Responsabilidade Principal	Exemplo
Model (Modelo)	Representa os dados e as regras de negócio . É responsável por acessar o banco de dados.	Classes que representam tabelas e métodos que manipulam dados.
View (Visão)	Define como os dados são exibidos para o usuário.	Páginas HTML, templates, APIs de resposta JSON.
Controller (Controlador)	Intermedia a comunicação entre o Model e a View. Recebe requisições, processa e retorna respostas.	Funções ou classes que tratam requisições HTTP.



Aula 30 — Introdução ao Padrão MVC + Análise de Sistema OO



MVC no Django

O **Django** segue um padrão **muito parecido com o MVC**, mas com **nomes ligeiramente diferentes**, conhecido como **MTV (Model–Template–View)**.

Apesar do nome mudar, a **função de cada parte é essencialmente a mesma**.

Conceito MVC	Equivalente em Django	Descrição
Model	<code>models.py</code>	Define a estrutura dos dados (tabelas, campos, relacionamentos).
View	<code>views.py</code>	Contém a lógica de controle: recebe requisições e retorna respostas.
Controller	<code>urls.py</code> + o próprio framework Django	Faz o roteamento das requisições para as views adequadas.
Template	<code>templates/</code>	É a camada de apresentação (HTML, CSS, JS).



Aula 30 — Introdução ao Padrão MVC + Análise de Sistema OO



Exemplo de estrutura simplificada de um app Django

```
meu_projeto/  
|  
├─ meu_app/  
|   ├── models.py      # Model: estrutura de dados (ex: classe Produto)  
|   ├── views.py       # View: lógica que trata as requisições  
|   ├── urls.py        # Controller: define as rotas (URLs)  
|   ├── templates/  
|   |   └─ produto_lista.html  
|   └─ admin.py        # Registro dos modelos no painel admin  
|  
├─ manage.py  
└─ meu_projeto/  
    ├── settings.py  
    ├── urls.py  
    └─ wsgi.py
```



Aula 30 — Introdução ao Padrão MVC + Análise de Sistema OO

Prática:

Refatorar um sistema simples de cadastro de produtos:

- Separar classes de dados (Model)
- Criar funções que interagem com o usuário (View)
- Criar controladores que gerenciam as ações (Controller)

```
sistema/  
├── model/  
│   └── produto.py  
├── view/  
│   └── interface.py  
├── controller/  
│   └── produto_controller.py  
└── main.py
```





Aula 30 — Introdução ao Padrão MVC + Análise de Sistema OO



Alguma dúvida ?

clara.gomes@ufpe.br



SOFTEX
PERNAMBUCO

 **Softex**

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO